HOCHSCHULE ESSLINGEN

Lab 1: Intro to RISC-V ASM Programming

Goals:

In Computer Architecture you will learn the architecture, programming & debugging of embedded systems: in the end you've developed a radio-frequency controlled clock based on the DCF77 signal.

In the first lab, You learn how to program & debug software using Assembler, laying the ground-work for the later labs: learning how to control the UI based on LEDs, a TFT display and buttons as input.

In Lab 2, you'll develop the core of the clock, using the system's timer module and input controls, to let the user set the initial time using the ADC

In Lab 3, you'll extend the clock with a DCF77 radio-frequency interface, receiving the current date and time information via antenna, so that the clock automatically sets time, switches between normal time and daylight savings time and tracks the date (with leap years) correctly.

Successful completion:

In each Lab You each must deliver & present working software. Additionally in Lab 2 and Lab 3 you must deliver a lab report with documented software. Please mark in Your source code, who worked on what parts of the code. The default is groups of **two persons**, any code copied (other than from the examples) must be marked.

Starting with Assembler

Preparation Task 1:

Write a short RISC-V assembler program based on template `lab1a`. The program shall:

- Count a 16 bit value in a loop starting from 0 counting up to 63 and then roll over,
- Blink the mono-color LED every other value (odd: LED on, even: LED off)
- The loop shall call a subroutine `delay_0_5sec`, which implements a time delay of 0.5sec.
  Hint: the subroutine shall not use global variables, as these would not work when the subroutine is used recursively. Global (read-only) constants are not critical.

Test your program in the debugger.

Assembler Utility functions

Preparation Task 2.1:

1. Based on the template `lab1b` create a new file `stringfct.S` to implement the function

```
int my_strtolower(char * string)
```

This subroutine shall convert a NUL-terminated ASCII string (ASCIIZ) from upper case to lower case characters. The string may contain a mix of upper and lower case characters as well as numbers and special characters like commas and dots:

- The conversion algorithm may use a trick: in ASCII, the upper and lower case alphabets are arranged in two blocks, where the only difference is bit5, being 0 for upper case and 1 for lower case characters. The conversion can be done using a bitwise OR operation:
  $\texttt{ASCII-Code}_{\texttt{Lower_case_character}} = \texttt{ASCII-Code}_{\texttt{Upper_case_character}} \mid (0x1 << 5);$

- Alternatively one could use the fact, that upper and lower case character ASCII code values use a constant difference:
  $\texttt{ASCII-Code}_{\texttt{Lower_case_character}} = \texttt{ASCII-Code}_{\texttt{Upper_case_character}} + 32_D;$
- The subroutine gets a pointer to the string in register `a0` when called. The string itself is in a global array in RAM and shall be modified by the subroutine via the pointer in register `a0`. It returns the length of the string in register `a0`.
- From the perspective of the calling code, the function must not change CPU registers `ra`, `s0/fp` up to `s11` and must not use any global variables.

2. Write another subroutine in `stringfct.S`:

```
int my_strlen(char * string)
```

This function returns the number of characters (excluding the final NUL Byte) of `string` (register `a0`) as return value (again in register `a0`).

3. Write another subroutine in Assembler in `stringfct.S`:

```
void my_strcpy(char * destination, char * source)
```

which copies the ASCIIZ string from memory pointed to by parameter `source` (in register `a1`) into memory pointed to by parameter `destination` (in register `a0`). The memory area of the destination in RAM must provide enough space to hold the string including the terminating zero.

4. Write a test program `main.S` for all subroutines. Use an initialized string in ROM, call `my_strlen`, then copy it to RAM using `my_strcpy`, before calling `my_strtolower`.

Carefully design the program using **flow-charts** for the `my_strtolower` subroutine and for the main test program, before you start writing code. Use the debugger to test your program.

Preparation Task 2.2:

Write a subroutine in RISC-V assembler into a new file `hexToASCII.S`:

```
void hexToASCII(char * string, unsigned int val)
```

which converts a 32-bit hexadecimal value in parameter `val` (passed in register `a0`) into a NUL-terminated ASCII-string. Example: For value $F018 a string consisting of ASCII values “F”, “0”, “1” and “8”, i.e. 46_H, 30_H, 31_H and 38_H as well as a terminating 00_H shall be returned. The maximum of eight hex digits shall be preceded by “0x” resulting in the string “0x0000F018” followed by NUL, aka in total 11 bytes.

- The 32 bit variable `val` is passed in register `a1`
- The pointer `string` is passed in register `a0` and points to a location in RAM, large enough to hold the result
- The conversion of each hexadecimal digit (4bit, aka nibble) into its ASCII-code may use a constant table, aka in ROM:

  ```
  .equ H2A,  "0123456789ABCDEF"
  ```

  The table is read via the hexadecimal digit as index into the array. The constant H2A shall be defined as global constant in the same file as the subroutine.
  The C-style pseudo code for this algorithm could be written as:

```
string[0] = '0';              // Place "0x" at beginning
string[1] = 'x';
string[2]  = H2A[(val >> 28) & 0xF];      // 16^7 digit → ASCII
string[3]  = H2A[(val >> 24) & 0xF];      // 16^6 digit → ASCII
      …
string[9]  = H2A[ val        & 0xF];      // 16^0 digit → ASCII
string[10] = '\0';                        // Terminating NUL char
```

- From the perspective of the calling code, the subroutine must not change CPU registers.

Write a `main.asm` to test the subroutine `hexToASCII()` by incrementing a 16 bit counter in a loop, calling `hexToASCII()` with the counter.
In the debugger, check for the "corner cases" 0, 1, …, 15, 16, …, 255, 256, 257, … , 65535 and finally the wrapping into 0, so that proper ASCII strings are generated.

Preparation Task 2.3:

Write a subroutine in RISC-V assembler into a new file `decToASCII.S`:

```
void decToASCII(char * string, int val)
```

This subroutine shall convert a 32 bit signed decimal value in parameter `val` (passed in register `a1`) into a NUL-terminated ASCII-string passed in register `a0`, which represents the decimal value:

- The 32-bit variable `val` is passed in register `a1`
- The pointer `string` is passed in register `a0` and stored in RAM, passed by the calling program. Don't forget to reserve enough space for the sign, all digits and the terminating NUL byte.
- Negative values shall be preceded by a '-' character, positive values by a ' ' (space) character. Values shall be right aligned.
  The C-style pseudo code for this algorithm:

```
if (val >= 0)       { string[0] = ' '; }   // Check sign of val
else                { string[0] = '-';     // Convert negative to
                      val = -val;          // positive value
(Q,R) = val / 10^9; string[1]=Q+'0';       // 10^9 digit→ASCII
(Q,R) = val / 10^8; string[2]=Q+'0';       // 10^8 digit→ASCII
(Q,R) = val / 10^7; string[3]=Q+'0';       // 10^7 digit→ASCII
(Q,R) = val / 10^6; string[4]=Q+'0';       // 10^6 digit→ASCII
(Q,R) = val / 10^5; string[5]=Q+'0';       // 10^5 digit→ASCII
(Q,R) = val / 10^4; string[6]=Q+'0';       // 10^4 digit→ASCII
(Q,R) = val / 10^3; string[7]=Q+'0';       // 10^3 digit→ASCII
(Q,R) = val / 10^2; string[8]=Q+'0';       // 10^2 digit→ASCII
(Q,R) = val / 10^1; string[9]=Q+'0';       // 10^1 digit→ASCII
                    string[10]=R+'0';      // 10^0 digit→ASCII
                    string[11]=0;          // Terminating NUL
```

  Note: (Q, R) = A / B → Q = string result of division A/B, R = remainder (german Rest) of integer division. Use the instruction `div` for the integer division and `rem` for the integer remainder. The `R+'0'` is adding the ASCII character `'0'` (aka 0x30, aka 48_D) to the remainder. This is an alternative method to convert a decimal digit rather than addressing the array via `H2A[R]`.
- From the perspective of the calling code, the subroutine must not change CPU register `ra`, `s0`/`fp`, `s1` … `s11`. The subroutine must not access global variables.

Write a `main.S` file to test this subroutine by incrementing a 32-bit signed counter in a loop, calling `decToASCII()` with this counter. Carefully design the program using program flow charts for the subroutine and for the test program before you start writing code.
In the debugger, check for the "corner cases" -2.147.483.648, -2.147.483.647, …, -1, 0, 1, 2, 100, 255, 256, …, 2.147.483.647 and the wrap-around to -2.147.483.648!
Do test both subroutines hexToASCII and decToASCII carefully. These subroutines will be used heavily in later programs. Bugs detected early will save you a lot of time and annoying debugging in later labs.

Appendix A: Code Comments and Interface Descriptions for Subroutines

Your code must have a reasonable amount of comments. Comments shall describe what the code is **supposed to** do on a per-block basis, i.e. describe complete code blocks but not individual statements.

	Bad style:	**Good style:**
`li s2, 0`	`# load immediate 0 into s2`	`# Initialize the counter`
`andi a0, a0, 0x1f`	`# and immediate in a0`	`# Consider values 0..31`
`beq a0, x0, next`	`# Branch if equal`	`# If NUL Byte, goto next`
`sb a0, 0(s1)`	`# Store Byte`	`# Store next character`
…		
`next:`	`# next`	`# Handle next character`

Additionally to commenting code blocks, you shall provide an interface description for all subroutines. The interface description **must** contain:

- A short description of the purpose of the function
- For assembly functions:
  - A list of all calling parameters (including their type and value range), where the parameter is placed (register or stack?)
  - Return parameter(s) of the subroutine (register?)
  - Which registers are modified by the subroutine (from the caller's perspective)
- For C functions:
  - Meanings of special values (e.g. `int` interpreted as Boolean, 0 being false and 1 true)
  - Range limits for function parameters, e.g. `int hours // 0 .. 23 only`
- Parameter and error checks done inside of the function, if any.

The interface description is inserted as comments directly above the subroutine's code in the program module. For many programming languages like Doxygen or JavaDoc extract comments and automatically generate suitable and portable documentation.

Examples for C and Assembler using Doxygen:

```
/**
 * \fn void led_off(int gpio);
 * \brief Turn off the LED (unset the GPIO output)
 *
 * Sets the given gpio port to 0.
 *
 * \param[in] gpio The gpio port (in register a0) to set to 0.
 *                 Value gpio must be between 0 and 31 (on RP2350)
 * \return void
 */
```